Jaypee Institute of Information Technology, Noida

PROJECT SYNOPSIS

Algorithmic Document Similarity Analyzer

Submitted to:

Dr Aastha Maheshwari

Dr Kirti Jain

Dr Rajiv Mishra

Dr Taj Alam

Dr Tarun Agrawal

Submitted By:

Shreyansh Rajat 2401030020

Aman Saxena 2401030017

Aditya Pandey 2401030027

Darsh Tiwari 2401030011

Mayank Garg 2401030001

Course Name: Design and Analysis of Algorithms Lab

Program: BTech CSE

4th Semester, 2nd Year

Introduction

With the exponential growth of digital academic and online content, ensuring the originality of documents has become increasingly challenging. Students, researchers, and content creators now have access to vast repositories of information, making it easier to reproduce or adapt existing material. An **Algorithmic Document Similarity Analyzer** is a system designed to evaluate the similarity between a given query document and a collection (corpus) of source documents using structured algorithmic techniques.

The project illustrates how fundamental techniques in string processing, efficient searching, hashing, indexing, and dynamic programming can be combined to build a scalable and accurate document comparison engine. The system follows a multi-stage plan that performs:

1. Text preprocessing and indexing
2. Exact phrase matching
3. Paraphrase detection using dynamic programming
4. Efficient corpus search
5. Ranking and reporting of results

Each stage is implemented using algorithms from the DAA syllabus, making the system both educational and functional.

Problem Statement

With the rapid growth of digital academic and textual content, manually comparing documents to determine similarity has become impractical and inefficient. Educational institutions, research environments, and content management systems require automated tools that can evaluate textual similarity accurately and efficiently across large collections of documents.

The problem addressed in this project is the design and implementation an Algorithmic Document Similarity Analyzer that can:

- Accept a query document
- Compare it against a corpus of source documents
- Detect exact and paraphrased copied content
- Generate similarity scores
- Rank the most similar documents

Brief Description of the Project and Features

The system accepts a document and compares it against a pre-loaded corpus of source documents. It identifies copied or similar text segments and produces a similarity report.

Core Features

1. **Text Preprocessing**
    - Converts text to lowercase
    - Removes punctuation and stop words
    - Tokenizes text into words
    - Generates n-grams (word shingles)

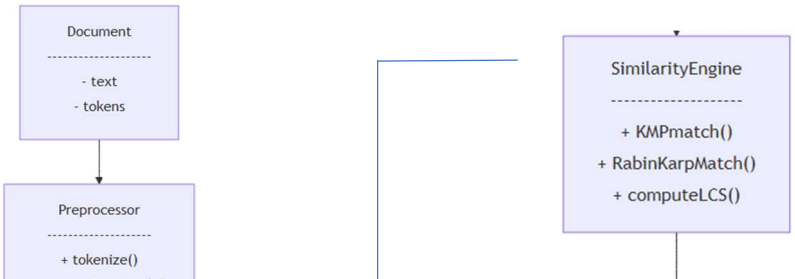
2. **Exact Phrase Matching**
    - Uses **KMP algorithm** for single-pattern matching
    - Uses **Rabin–Karp algorithm** for multi-pattern matching
    - Detects verbatim copied passages

3. **Longest Common Substring Detection**
    - Uses **Suffix Arrays with LCP**
    - Identifies longest shared contiguous text segments

4. **Paraphrase Detection**
    - Uses **Longest Common Subsequence (LCS)**
    - Detects similarity despite word reordering or substitution

5. **Efficient Corpus Search**
    - Uses **Divide and Conquer strategy**
    - Uses **Binary Search on sorted fingerprint index**
    - Prunes irrelevant documents early

Design and Class Diagram



Conclusion

The Plagiarism Detection Engine demonstrates how classical algorithms can be integrated into a complete real-world system. Every stage of the pipeline is built using core topics from the Design and Analysis of Algorithms syllabus, including sorting, searching, string matching, dynamic programming, greedy methods, and divide-and-conquer strategies.

The project not only detects plagiarism effectively but also serves as a practical demonstration of how fundamental algorithmic concepts are applied in real-world systems such as search engines, plagiarism checkers, and version control tools.

References

- **KMP Algorithm for Pattern Searching**
  https://www.geeksforgeeks.org/dsa/kmp-algorithm-for-pattern-searching/

- **Rabin–Karp Algorithm for Pattern Searching**
  https://www.geeksforgeeks.org/rabin-karp-algorithm-for-pattern-searching/

- **Suffix Array – Introduction**
  https://www.geeksforgeeks.org/suffix-array-set-1-introduction/

- **Trie – Insert and Search**
  https://www.geeksforgeeks.org/trie-insert-and-search/

- **Longest Common Subsequence (LCS)**
  https://www.geeksforgeeks.org/longest-common-subsequence-dp-4/